Check for updates

THE INTERNATIONAL JOURNAL of HIGH PERFORMANCE COMPUTING APPLICATIONS

Article

The International Journal of High Performance Computing Applications
25(4) 496–509
© The Author(s) 2011
Reprints and permission:
sagepub.co.uk/journalsPermissions.nav
DOI: 10.1177/1094342011403516
hpc.sagepub.com
SAGE

The Combinatorial BLAS: design, implementation, and applications

Aydın Buluç[1] **and John R Gilbert**[2]

Abstract

This paper presents a scalable high-performance software library to be used for graph analysis and data mining. Large combinatorial graphs appear in many applications of high-performance computing, including computational biology, informatics, analytics, web search, dynamical systems, and sparse matrix methods. Graph computations are difficult to parallelize using traditional approaches due to their irregular nature and low operational intensity. Many graph computations, however, contain sufficient coarse-grained parallelism for thousands of processors, which can be uncovered by using the right primitives. We describe the parallel Combinatorial BLAS, which consists of a small but powerful set of linear algebra primitives specifically targeting graph and data mining applications. We provide an extensible library interface and some guiding principles for future development. The library is evaluated using two important graph algorithms, in terms of both performance and ease-of-use. The scalability and raw performance of the example applications, using the Combinatorial BLAS, are unprecedented on distributed memory clusters.

Keywords

Betweenness centrality, combinatorial BLAS, combinatorial scientific computing, graph analysis, Markov clustering, mathematical software, parallel graph library, software framework, sparse matrices

1 Introduction

Large scale software development is a formidable task that requires an enormous amount of human expertise, especially when it comes to writing software for parallel computers. Writing every application from scratch is an unscalable approach given the complexity of the computations and the diversity of the computing environments involved. Raising the level of abstraction of parallel computing by identifying the algorithmic commonalities across applications is becoming a widely accepted path to solution for the parallel software challenge (Asanovic et al., 2006; Brodman et al., 2009). Primitives both allow algorithm designers to think on a higher level of abstraction, and help to avoid duplication of implementation efforts.

Primitives have been successfully used in the past to enable many computing applications. The *Basic Linear Algebra Subroutines* (BLAS) for numerical linear algebra (Lawson et al., 1979) are probably the canonical example of a successful primitives package. The BLAS became widely popular following the success of LAPACK (Anderson et al., 1992). LINPACK's use of the BLAS encouraged experts (preferably the hardware vendors themselves) to implement its vector operations for optimal performance. In addition to efficiency benefits, BLAS offered portability by providing a common interface. It also indirectly encouraged structured programming. Most of the reasons for developing the BLAS package about four decades ago are generally valid for primitives today.

In contrast to numerical computing, a scalable software stack that eases the application programmer's job does not exist for computations on graphs. Some existing primitives can be used to implement a number of graph algorithms. Scan primitives (Blelloch, 1990) are used for solving the maximum flow, minimum spanning tree, maximal independent set, and (bi)connected components problems efficiently. On the other hand, it is possible to implement some clustering and connected components algorithms using the MapReduce model (Dean and Ghemawat, 2008), but the approaches are quite unintuitive and the performance is unknown (Cohen, 2009). Our work fills a crucial gap by providing primitives that can be used for traversing graphs.

The goal of having a BLAS-like library for graph computation is to support rapid implementation of graph

[1]High Performance Computing Research, Lawrence Berkeley National Laboratory, USA
[2]Department of Computer Science, University of California, USA

Corresponding author:
Aydin Buluç, High Performance Computing Research, Lawrence Berkeley National Laboratory, 1 Cyclotron Road, Berkeley, CA 94720, USA
Email: abuluc@lbl.gov

algorithms using a small yet important subset of linear algebra operations. The library should also be parallel and scale well due to the massive size of graphs in many modern applications.

The Matlab reference implementation of the HPCS Scalable Synthetic Compact Applications graph analysis (SSCA#2) benchmark (Bader et al., a) was an important step towards using linear algebra operations for implementing graph algorithms. Although this implementation was a success in terms of expressibility and ease of implementation, its performance was about 50% worse than the best serial implementation. Mostly, the slowdown was due to limitations of Matlab for performing integer operations. The parallel scaling was also limited on most parallel Matlab implementations.

In this paper we introduce a scalable high-performance software library, the Combinatorial BLAS, to be used for graph computations on distributed memory clusters. The Combinatorial BLAS is intended to provide a common interface for high-performance graph kernels. It is unique among other graph libraries for combining scalability with distributed memory parallelism, which is partially achieved through ideas borrowed from the domain of parallel numerical computation. Our library is especially useful for tightly-coupled, traversal-based computations on graphs.

The success of the Combinatorial BLAS partially relies on the promise of future optimizations. Although specific optimizations on various architectures require extensive research, working at the right granularity is a prerequisite for successful incorporation of future optimizations. Our paper does not just provide an API for graph computations; it also presents a case study for a specific architecture (distributed memory systems). The scalability of our reference implementation serves as a proof of concept that Combinatorial BLAS routines have the right granularity.

The remainder of this paper is organized as follows. Section 2 summarizes existing frameworks for parallel graph and sparse matrix computations. Section 3 describes the design and the guiding principles of the Combinatorial BLAS library. Section 4 gives an overview of the software engineering techniques used in the implementation. Section 5 presents performance results from two important graph applications implemented using the Combinatorial BLAS primitives. Section 6 offers some concluding remarks as well as future directions.

2 Related work and comparison

2.1 Graph primitives

In the introduction, we mentioned some general primitives with applications on graph computations. In this section we discuss two classes of primitives that are designed specifically for graph algorithms, namely the sparse array-based primitives used in the Combinatorial BLAS and the alternative visitor-based primitives.

Sparse array-based primitives leverage the duality between graphs and sparse matrices. The adjacency matrix of the graph is considered as a sparse matrix data structure, and linear algebraic operations on this matrix map to certain graph operations. This approach encapsulates problems of load balancing, synchronization and latency in the implementation of the primitives. Furthermore, whenever available, it seamlessly exploits multiple levels of parallelism as shown in our betweenness centrality implementation in Section 5.1.

Previous work (Gilbert et al., 2008; Fineman and Robinson, 2011) has shown that purely linear algebraic primitives are surprisingly powerful for representing a wide variety of graph algorithms. However, the same studies showed that a certain class of algorithms that make heavy use of priority queues could not be represented with the same asymptotic complexity using algebraic primitives only. Examples of such algorithms include Prim's minimum spanning tree algorithm and Dijkstra's shortest paths algorithm. We plan to develop coarse-grained alternatives to priority queues that can operate efficiently on distributed sparse matrices. One promising approach involves reformulating such algorithms so that they work on relaxed nonlinearizable data structures (Kirsch et al., 2010) that expose more parallelism.

Visitor-based search patterns of the Boost Graph Library (Siek et al., 2001) and its relatives (Gregor and Lumsdaine, 2005; Berry et al., 2007) are powerful methods to express various graph algorithms, including many classical sequential algorithms that rely on priority queues and depth-first search. Visitor patterns, however, require fine-grained reasoning about synchronization and race conditions, and may be too fine-grained for a primitive to optimize across a large aggregation of operations. Describing operations one vertex or edge at a time can make the computation data driven and obstruct opportunities for optimization. These concerns are not unique to the visitor paradigm; operations such as cost-based priority searches or depth-first search seem to be inherently difficult to scale in parallel. Providing the users with an opaque operational space might also tempt them to use unscalable primitives.

2.2 Frameworks for parallel graph computation

This section surveys working implementations of graph computations, rather than research on parallel graph algorithms. We focus on frameworks and libraries instead of parallelization of stand-alone applications. The current landscape of software for graph computations is summarized in Table 1.

The Parallel Boost Graph Library (PBGL) (Gregor and Lumsdaine, 2005) is a parallel library for distributed memory computing on graphs. It is a significant step towards facilitating rapid development of high performance applications that use distributed graphs as their main data structure. Like the sequential Boost Graph Library (Siek et al., 2001), it has a dual focus on efficiency and flexibility.

Table 1. High-performance libraries and toolkits for parallel graph analysis

Library/Toolkit	Parallelism	Abstraction	Offering	Scalability
PBGL (Gregor and Lumsdaine, 2005)	Distributed	Visitor	Algorithms	Limited
GAPDT (Gilbert et al., 2008)	Distributed	Sparse Matrix	Both	Limited
MTGL (Berry et al., 2007)	Shared	Visitor	Algorithms	Unknown
SNAP (Bader and Madduri, 2008)	Shared	Various	Both	High
Pregel (Malewicz et al., 2010)	Distributed	Vertex-centric	None	Preliminary
Combinatorial BLAS	Distributed	Sparse Matrix	Kernels	High

It relies heavily on generic programming through C++ templates.

Lumsdaine et al. (2007) observed poor scaling of PBGL for some large graph problems. We believe that the scalability of PBGL is limited due to two main factors. First, the graph is distributed by vertices instead of edges, which corresponds to a one-dimensional partitioning in the sparse matrix world. For many graph kernels, distributed algorithms on 1D partitioned data requires communication to take place among all p processors, as opposed to groups of $\sqrt{p}$ processors with 2D partitioning. Secondly, the data-driven fine-grained nature of the computation that is characteristic of the visitor pattern (Section 2.1) might also limit performance.

The MultiThreaded Graph Library (MTGL) (Berry et al., 2007) was originally designed for development of graph applications on massively multithreaded machines, namely Cray MTA-2 and XMT. It was later extended to run on mainstream shared-memory architectures (Barrett et al., 2009). MTGL is a significant step towards an extensible and generic parallel graph library. As of now, only preliminary performance results are published for MTGL.

The Graph Algorithm and Pattern Discovery Toolbox (GAPDT, later renamed KDT) (Gilbert et al., 2008) provides both combinatorial and numerical tools to manipulate large graphs interactively. KDT runs sequentially on Matlab or in parallel on Star-P (Shah and Gilbert, 2004), a parallel dialect of Matlab. Although KDT focuses on algorithms, the underlying sparse matrix infrastructure also exposes linear algebraic kernels. KDT, like PBGL, targets distributed-memory machines. Differently from PBGL, it uses operations on distributed sparse matrices for parallelism. KDT provides an interactive environment instead of compiled code, which makes it unique among the frameworks surveyed here. Like PBGL, KDT's main weakness is limited scalability due to its one-dimensional distribution of sparse arrays.

The Small-world Network Analysis and Partitioning (SNAP) framework (Bader and Madduri, 2008) contains algorithms and kernels for exploring large-scale graphs. SNAP is a collection of different algorithms and building blocks that are optimized for small-world networks. It combines shared-memory thread-level parallelism with state-of-the-art algorithm engineering for high performance. The graph data can be represented in a variety of different formats depending on the characteristics of the algorithm that operates on it. SNAP's performance and scalability are high for the reported algorithms, but a head-to-head performance comparison with PBGL and KDT is not available.

Both MTGL and SNAP are powerful toolboxes for graph computations on multithreaded architectures. For future extensions, MTGL relies on the visitor concept it inherits from the PBGL, while SNAP relies on its own kernel implementations. Both software architectures are maintainable as long as the target architectures remain the same.

Algorithms on massive graphs with billions of vertices and edges require hundreds of gigabytes of memory. For a special purpose supercomputer such as XMT, memory might not be a problem; but commodity shared-memory architectures have limited memory. Thus, MTGL or SNAP will be likely to find limited use in commodity architectures without either distributed memory or out-of-core support. Experimental studies show that an out-of-core approach (Ajwani et al., 2007) is two orders of magnitude slower than an MTA-2 implementation for parallel breadth-first search (Bader and Madduri, 2006b). Given that many graph algorithms, such as clustering and betweenness centrality, are computationally intensive, out-of-core approaches are infeasible. Therefore, distributed memory support for running graph applications of general purpose computers is essential. Neither MTGL nor SNAP seems easily extensible to distributed memory.

Recently, Google introduced Pregel (Malewicz et al., 2010), which is a vertex-centric message passing system targeting distributed memory. It is intended for programs that can be described in the Bulk Synchronous Parallel (BSP) model (Valiant, 1990). In Pregel, edges are not first-class citizens and they can not impose computation. By contrast, the Combinatorial BLAS is edge-based; each element of the sparse matrix represents an edge, and the underlying semiring defines the computation on that edge. The BSP style allows Pregel to avoid fine-grained synchronization. It remains to be seen how its message passing model will affect programmer productivity.

2.3 Frameworks for parallel sparse matrix computation

We briefly mention some other work on parallel sparse arrays, much of which is directed at numerical sparse matrix computation rather than graph computation. We also mention the suitability of the Combinatorial BLAS for numerical computation.

Many libraries exist for solving sparse linear system and eigenvalue problems; some, like Trilinos (Heroux et al., 2005), include significant combinatorial capabilities. The Sparse BLAS (Duff et al., 2002) is a standard API for numerical matrix- and vector-level primitives; its focus is infrastructure for iterative linear system solvers, and therefore it does not include such primitives as sparse matrix–matrix multiplication and sparse matrix indexing. Global Arrays (Nieplocha et al., 2006) is a parallel dense and sparse array library that uses a one-sided communication infrastructure portable to message-passing, NUMA, and shared-memory machines. Star-P (Shah and Gilbert, 2004) and pMatlab (Kepner, 2009) are parallel dialects of Matlab that run on distributed-memory message-passing machines; both include parallel sparse distributed array infrastructures.

While the Combinatorial BLAS is designed with graph algorithms in mind, some of its functionality can be used for numerical computing as well. This is because Combinatorial BLAS primitives can operate on arbitrary semirings such as the $(+,*)$ ring, as they do not require idempotency of the underlying semiring for correctness. For example, the SpGEMM operation can be used to construct a coarser grid during the V-cycle of the Algebraic Multigrid (AMG) method (Briggs et al., 2000). The scalability of our SpGEMM implementation for this task was experimentally evaluated in our recent work (Buluç and Gilbert, 2010). However, the Combinatorial BLAS does not attempt to provide a complete set of primitives for implementing numerical algorithms.

3 Design philosophy

3.1 Overall design

The first class citizens of the Combinatorial BLAS are distributed sparse matrices. Application domain interactions that are abstracted into a graph are concretely represented as a sparse matrix. Therefore, all non-auxiliary functions are designed to operate on sparse matrix objects. There are three other types of objects that are used by some of the functions: dense matrices, dense vectors, sparse vectors. Concrete data structures for these objects are explained in detail in Section 4.

Our design is influenced by some of the guiding principles of the PETSc package (Balay et al., 1997), namely 'managing the communication in the context of higher-level operations on parallel objects' and 'aggregation of data for communication'. We are also influenced by some of the software engineering decisions made by the Trilinos project (Heroux et al., 2005), such as the use of compile-time polymorphism.

We define a common abstraction for all sparse matrix storage formats, making it possible to implement a new format and plug it in without changing the rest of the library. For scalability, as well as to avoid inter-library and intra-library collisions, matrices and vectors can be distributed over only a subset of processors by passing restricted MPI communicators to constructors. We do not attempt to create the illusion of a flat address space; communication is handled internally by parallel classes of the library. Likewise, we do not always provide storage independence due to our emphasis on high performance. Some operations have different semantics depending on whether the underlying object is sparse or dense.

The Combinatorial BLAS routines (API functions) are supported both sequentially and in parallel. The versions that operate on parallel objects manage communication and call the sequential versions for computation. This symmetry of function prototypes has a nice effect on interoperability. The parallel objects can just treat their internally stored sequential objects as black boxes supporting the API functions. Conversely, any sequential class becomes fully compatible with the rest of the library as long as it supports the API functions and allows access to its internal arrays through an adapter object. This decoupling of parallel logic from sequential parts of the computation is one of the distinguishing features of the Combinatorial BLAS.

3.2 The Combinatorial BLAS routines

We selected the operations to be supported by the API by a top-down, application driven process. Commonly occurring computational patterns in many graph algorithms are abstracted into a few linear algebraic kernels that can be efficiently mapped onto the architecture of distributed memory computers. A summary of the current API for the Combinatorial BLAS is shown in Table 2. The Matlab phrasings in Table 2 are examples, not complete translations, of the functionality. The API is not intended to be final and will be extended as more applications are analyzed and new algorithms are invented.

We address the tension between generality and performance by the zero overhead principle. Our primary goal is to provide work-efficiency for the targeted graph algorithms. The interface is kept general, simple, and clean so long as doing so does not add significant overhead to the computation. The guiding principles in the design of the API are listed below, each one illustrated with an example.

(1) If multiple operations can be handled by a single function prototype without degrading the asymptotic performance of the algorithm they are to be part of, then we provide a generalized single prototype. Otherwise, we provide multiple prototypes.

For elementwise operations on sparse matrices, although it is tempting to define a single function prototype that accepts a *binop* parameter, the most-efficient data access pattern depends on the binary operation. For instance, ignoring numerical cancellation, elementwise addition is most efficiently implemented as a union of two sets, while multiplication is the intersection. If it proves to be efficiently implementable (using either function object traits

Table 2. Summary of the current API for the Combinatorial BLAS

Function	Applies to	Parameters		Returns	Matlab Phrasing
SpGEMM	Sparse Matrix (as friend)	**A**, **B**: trA: trB:	sparse matrices transpose **A** if true transpose **B** if true	Sparse Matrix	$\mathbf{C} = \mathbf{A} * \mathbf{B}$
SpMV	Sparse Matrix (as friend)	**A**: **x**: trA:	sparse matrices sparse or dense vector(s) transpose **A** if true	Sparse or Dense Vector(s)	$\mathbf{y} = \mathbf{A} * \mathbf{x}$
SpEWiseX	Sparse Matrices (as friend)	**A**, **B**: notA: notB:	sparse matrices negate **A** if true negate **B** if true	Sparse Matrix	$\mathbf{C} = \mathbf{A} * \mathbf{B}$
Reduce	Any Matrix (as method)	dim: *binop*:	dimension to reduce reduction operator	Dense Vector	sum(**A**)
SpRef	Sparse Matrix (as method)	**p**: **q**:	row indices vector column indices vector	Sparse Matrix	$\mathbf{B} = \mathbf{A}(\mathbf{p}, \mathbf{q})$
SpAsgn	Sparse Matrix (as method)	**p**: **q**: **B**:	row indices vector column indices vector matrix to assign	none	$\mathbf{A}(\mathbf{p}, \mathbf{q}) = \mathbf{B}$
Scale	Any Matrix (as method)	**rhs**:	any object (except a sparse matrix)	none	Check guiding principles 3 and 4
Scale	Any Vector (as method)	**rhs**:	any vector	none	none
Apply	Any Object (as method)	*unop*:	unary operator (applied to non-zeros)	None	none

or run-time type information), all elementwise operations between two sparse matrices may have a single function prototype in the future.

On the other hand, the data access patterns of matrix–matrix and matrix–vector multiplications are independent of the underlying semiring. As a result, the sparse matrix–matrix multiplication routine (SpGEMM) and the sparse matrix–vector multiplication routine (SpMV) each have a single function prototype that accepts a parameter representing the semiring, which defines user-specified addition and multiplication operations for SpGEMM and SpMV.

(2) *If an operation can be efficiently implemented by composing a few simpler operations, then we do not provide a special function for that operator.*

For example, making a non-zero matrix **A** column stochastic can be efficiently implemented by first calling Reduce on **A** to get a dense row vector **v** that contains the sums of columns, then obtaining the multiplicative inverse of each entry in **v** by calling the Apply function with the unary function object that performs $f(v_i) = 1/v_i$ for every v_i it is applied to, and finally calling Scale(**v**) on **A** to effectively divide each non-zero entry in a column by its sum. Consequently, we do not provide a special function to make a matrix column stochastic.

On the other hand, a commonly occurring operation is to zero out some of the non-zeros of a sparse matrix. This often comes up in graph traversals, where $\mathbf{X}^k$ represents the kth frontier, the set of vertices that are discovered during the kth iteration. After the frontier expansion $\mathbf{A}^{\mathrm{T}}\mathbf{X}^k$, previously discovered vertices can be pruned by performing an elementwise multiplication with a matrix $\mathbf{Y}$ that includes a zero for every vertex that has been discovered before, and non-zeros elsewhere. However, this approach might not be work-efficient as $\mathbf{Y}$ will often be dense, especially in the early stages of the graph traversal.

Consequently, we provide a generalized function SpEWiseX that performs the elementwise multiplication of sparse matrices $op(\mathbf{A})$ and $op(\mathbf{B})$. It also accepts two auxiliary parameters, notA and notB, that are used to negate the sparsity structure of **A** and **B**. If notA is true, then $op(\mathbf{A})(i,j) = 0$ for every non-zero $\mathbf{A}(i,j) \neq 0$, and $op(\mathbf{A})(i,j) = 1$ for every zero $\mathbf{A}(i,j) = 0$. The role of notB is identical. Direct support for the logical NOT operations is crucial to avoid the explicit construction of the dense $not(\mathbf{B})$ object.

(3) *To avoid expensive object creation and copying, many functions also have in-place versions. For operations that can be implemented in place, we deny access to any other variants only if those increase the running time.*

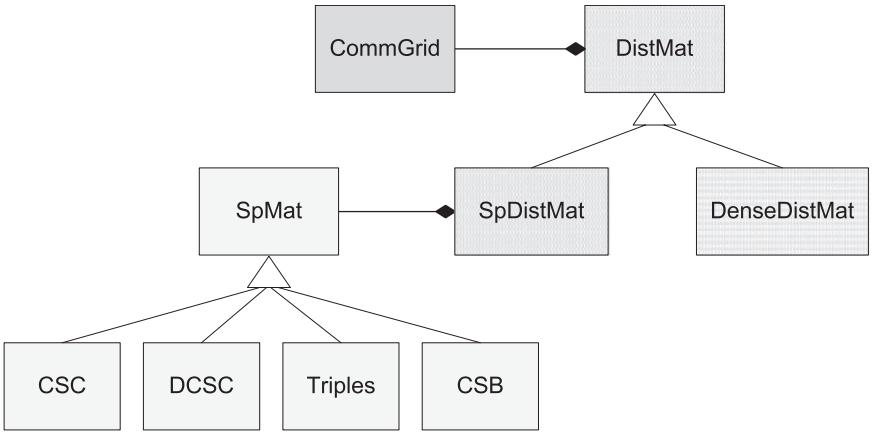


Figure 1. Software architecture for matrix classes

For example, SCALE(**B**) is a member function of the sparse matrix class that takes a dense matrix as a parameter. When called on the sparse matrix **A**, it replaces each $\mathbf{A}(i,j) \neq 0$ with $\mathbf{A}(i,j) \cdot \mathbf{B}(i,j)$. This operation is implemented only in-place because $\mathbf{B}(i,j)$ is guaranteed to exist for a dense matrix, allowing us to perform a single scan of the non-zeros of **A** and update them by doing fast lookups on **B**. Not all elementwise operations can be efficiently implemented in-place (for example elementwise addition of a sparse matrix and a dense matrix will produce a dense matrix), so we declare them as members of the dense matrix class or as global functions returning a new object.

(4) *In-place operations have slightly different semantics depending on whether the operands are sparse or dense. In particular, the semantics favor leaving the sparsity pattern of the underlying object intact as long as another function (possibly not in-place) handles the more conventional semantics that introduces/deletes non-zeros.*

For example, SCALE is an overloaded method, available for all objects. It does not destroy sparsity when called on sparse objects and it does not introduce sparsity when called on dense objects. The semantics of the particular SCALE method are dictated by its the class object and its operand. Called on a sparse matrix **A** with a vector **v**, it independently scales non-zero columns (or rows) of the sparse matrix. For a row vector **v**, SCALE replaces every non-zero $\mathbf{A}(i,j)$ with $\mathbf{v}(j) \cdot \mathbf{A}(i,j)$. The parameter **v** can be dense or sparse. In the latter case, only a portion of the sparse matrix is scaled. That is, $\mathbf{v}(j)$ being zero for a sparse vector does not zero out the corresponding jth column of **A**. The SCALE operation never deletes columns from **A**; deletion of columns is handled by the more expensive SPASGN function described below. Alternatively, zeroing out columns during scaling can be accomplished by performing $\mathbf{A} \cdot \mathrm{DIAG}(\mathbf{v})$ with a sparse **v**. Here, DIAG(**v**) creates a sparse matrix with diagonal populated from the elements of **v**. Note that this alternative approach is still more expensive than SCALE, as the multiplication returns a new matrix.

SPASGN and SPREF are generalized sparse matrix assignment and indexing operations. They are very powerful primitives that take vectors **p** and **q** of row and column indices. When called on the sparse matrix **A**, SPREF returns a new sparse matrix whose rows are the $\mathbf{p}(i)$th rows of **A** for $i = 0, \ldots, length(\mathbf{p}) - 1$ and whose columns are the $\mathbf{q}(j)$th columns of **A** for $j = 0, \ldots, length(\mathbf{q}) - 1$. SpAsgn has similar syntax, except that it returns a reference (a modifiable lvalue) to some portion of the underlying object as opposed to returning a new object.

4 A reference implementation

4.1 The software architecture

In our reference implementation, the main data structure is a distributed sparse matrix object ***SpDistMat*** which HAS-A local sparse matrix that can be implemented in various ways as long as it supports the interface of the base class ***SpMat***. All features regarding distributed-memory parallelization, such as the communication patterns and schedules, are embedded into the distributed objects (sparse and dense) through the ***CommGrid*** object. Global properties of distributed objects, such as the total number of nonzeros and the overall matrix dimensions, are not explicitly stored. They are computed by reduction operations whenever necessary. The software architecture for matrices is illustrated in Figure 1. Although the inheritance relationships are shown in the traditional way (via inclusion polymorphism as described by Cardelli and Wegner (1985)), the class hierarchies are static, obtained by parameterizing the base class with its subclasses as explained below.

To enforce a common interface as defined by the API, all types of objects derive from their corresponding base classes. The base classes only serve to dictate the interface. This is achieved through static object oriented

```
// Abstract base class for all derived serial sparse matrix classes
// Has no data members, copy constructor or assignment operator
// Uses static polymorphism through curiously recurring templates
// Template parameters:
//  IT (index type), NT (numerical type), DER (derived class type)
template <class IT, class NT, class DER>
class SpMat
{
    typedef SpMat<IT,NT,DER> SpMatIns;
public:
    // Standard destructor, copy constructor and assignment
    //  are generated by compiler, they all do nothing
    // Default constructor also exists, and does nothing more
    //  than creating Base<Derived>() and Derived() objects
    // One has to call the Create function to get a nonempty object
    void Create (const vector<IT>& essentials);

    SpMatIns operator() (const vector<IT>& ri, const vector<IT>& ci);

    template <typename SR>       // SR: Semiring object
    void SpGEMM (SpMatIns & A, SpMatIns & B, bool TrA, bool TrB);

    template <typename NNT>      // NNT: New numeric type
    operator SpMatIns() const;

    void Split (SpMatIns & partA, SpMatIns & partB);
    void Merge (SpMatIns & partA, SpMatIns & partB);

    Arr<IT,NT> GetArrays() const;
    vector<IT> GetEssentials() const;

    void Transpose();

    bool operator== (const SpMatIns & rhs) const;

    ofstream& put(ofstream& outfile) const;
    ifstream& get(ifstream& infile);

    bool isZero() const;
    IT getnrow() const;
    IT getncol() const;
    IT getnnz() const;
}
```

Figure 2. Partial C++ interface of the base SpMat class

programming (OOP) techniques (Burrus et al., 2003) rather than expensive dynamic dispatch. A trick known as the *Curiously Recurring Template Pattern* (CRTP), a term coined by Coplien (1995), emulates dynamic dispatch statically, with some limitations. These limitations, such as the inability to use heterogeneous lists of objects that share the same type class, however, are not crucial for the Combinatorial BLAS. In CRTP, the base class accepts a template parameter of the derived class.

The SpMat base class implementation is given as an example in Figure 2. As all exact types are known at compile time, there are no runtime overheads arising from dynamic dispatch. In the presence of covariant arguments, static polymorphism through CRTP automatically allows for better type checking of parameters. In the SpGEMM example, with classical OOP, one would need to dynamically inspect the actual types of **A** and **B** to see whether they are compatible and to call the right subroutines. This requires run-time type information queries and *dynamic_cast()* operations. Also, relying on run-time operations is unsafe, as any unconforming set of parameters will lead to a run-time error or an exception. Static OOP catches any such incompatibilities at compile time. An equally expressive alternative to CRTP would be to use enable_if-based function overloads (Järvi et al., 2003).

The SpMat object is local to a node but it need not be sequential. It can be implemented as a shared-memory data structure, amenable to thread-level parallelization. This flexibility will allow future versions of the Combinatorial BLAS algorithms to support hybrid parallel programming. The distinguishing feature of SpMat is contiguous storage of its sparse matrix, making it accessible by all other components (threads/processes). In this regard it is different from the SpDistMat, which distributes the storage of its sparse matrices.

Almost all popular sparse matrix storage formats are internally composed of a number of arrays (Dongarra, 2000; Saad, 2003; Buluç et al., 2009), since arrays are cache friendlier than pointer-based data structures. Following this observation, the parallel classes handle object creation and communication through what we call an ***Essentials*** object, which is an adapter for the actual sparse

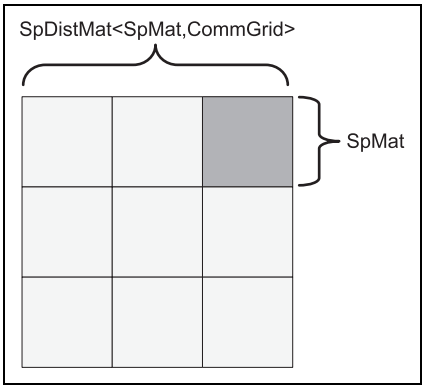


Figure 3. Distributed sparse matrix class and storage

matrix object. The Essentials of a sparse matrix object includes its dimensions, number of non-zeros, starting addresses of its internal arrays and the sizes of those arrays.

The use of Essentials allows any SpDistMat object to have any SpMat object internally. For example, communication can be overlapped with computation in the SpGEMM function by prefetching the internal arrays through one-sided communication. Alternatively, another SpDistMat class that uses a completely different communication library, such as GASNet (Bonachea, 2002) or ARMCI (Nieplocha et al., 2005), can be implemented without requiring any changes to the sequential SpMat object.

Most combinatorial operations use more than the traditional floating-point arithmetic, with integer and boolean operations being prevalent. To provide the user the flexibility to use matrices and vectors with any scalar type, all of our classes and functions are templated. A practical issue is to be able to perform operations between two objects holding different scalar types, e.g. multiplication of a boolean sparse matrix by an integer sparse matrix. Explicit upcasting of one of the operands to a temporary object might have jeopardized performance due to copying of such big objects. The template mechanism of C++ provided a neat solution to the mixed mode arithmetic problem by providing automatic type promotion through trait classes (Barton and Nackman, 1994). Arbitrary semiring support for matrix–matrix and matrix–vector products is allowed by passing a class (with static **add** and **multiply** functions) as a template parameter to corresponding SpGEMM and SpMV functions.

4.2 Management of distributed objects

The processors are logically organized as a two-dimensional grid in order to limit most of the communication to take place along a processor column or row with $\sqrt{p}$ processors, instead of communicating potentially with all p processors. The partitioning of distributed matrices (sparse and dense) follows this processor grid organization, using a 2D block decomposition, also called the checkerboard partitioning (Grama et al., 2003). Figure 3 shows this for the sparse case.

Portions of dense matrices are stored locally as two-dimensional dense arrays in each processor. Sparse matrices (SpDistMat objects), on the other hand, have many possible representations, and the right representation depends on the particular setting or the application. We (Buluç and Gilbert, 2008b, 2010) previously reported the problems associated with using the popular compressed sparse rows (CSR) or compressed sparse columns (CSC) representations in a 2D block decomposition. The triples format does not have the same problems but it falls short of efficiently supporting some of the fundamental operations. Therefore, our reference implementation uses the DCSC format, explained in detail by Buluç and Gilbert (2008b). As previously mentioned, this choice is by no means exclusive and one can replace the underlying sparse matrix storage format with his or her favorite format without needing to change other parts of the library, as long as the format implements the fundamental sequential API calls mentioned in the previous section.

Vectors can be distributed either to all the processors of the 2D grid, or only to those processors on the diagonal. While the diagonal-only distribution is conceptually simpler and achieves symmetric performance for matrix vector and vector-matrix multiplications, fully distributing the vector to all the processors is more scalable and memory-efficient. The Combinatorial BLAS offers both alternatives. The high-level structure and parallelism of sparse and dense vectors are the same, the only difference being how the local data is stored in processors. A dense vector naturally uses a dense array, while a sparse vector is internally represented as a list of index–value pairs.

5 Applications and performance analysis

This section presents two applications of the Combinatorial BLAS library. We report the performance of two algorithms on distributed-memory clusters, implemented using the Combinatorial BLAS primitives. The code for these applications, along with an alpha release of the complete library, can be freely obtained from http://gauss.cs.ucsb.edu/code/index.shtml.

5.1 Betweenness centrality

Betweenness centrality (Freeman, 1977), a centrality metric based on shortest paths, is the main computation on which we evaluate the performance of our proof-of-concept implementation of the Combinatorial BLAS. There are two reasons for this choice. First, it is a widely-accepted metric that is used to quantify the relative importance of vertices in the graph. The betweenness centrality (BC) of a vertex is the normalized ratio of the number of shortest paths that pass through a vertex to the total number of shortest paths in the graph. This is formalized in Equation 1, where σ_{st} denotes the number of shortest paths from s to t, and $\sigma_{st}(v)$ is the number of such paths passing through vertex v.

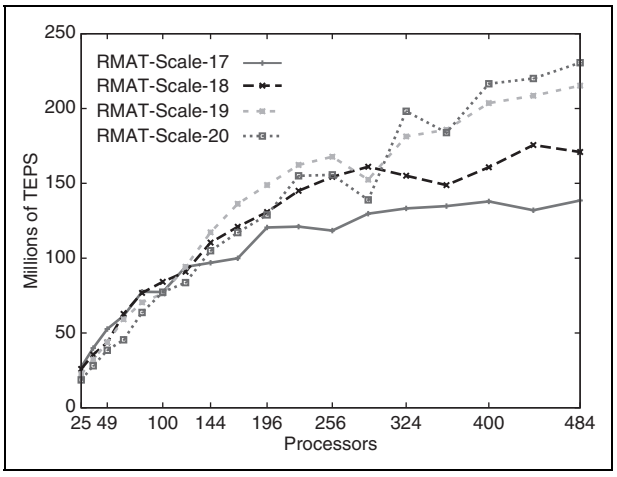


Figure 4. Parallel strong scaling of the distributed-memory betweenness centrality implementation (smaller input sizes)

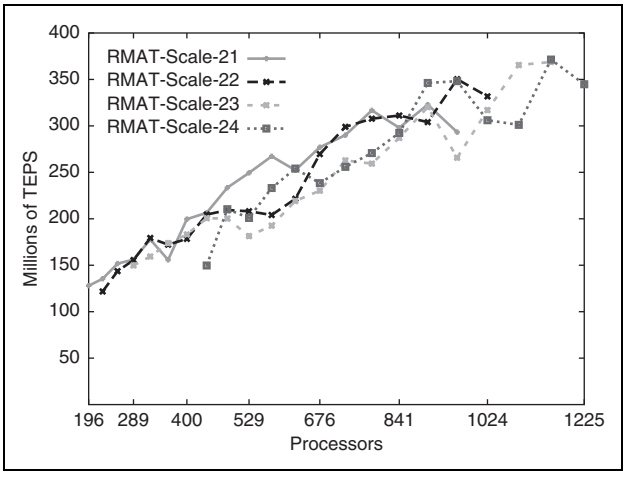


Figure 5. Parallel strong scaling of the distributed-memory betweenness centrality implementation (bigger input sizes)

$$BC(v) = \sum_{s \neq v \neq t \in V} \frac{\sigma_{st}(v)}{\sigma_{st}} \quad (1)$$

A vertex v with a high betweenness centrality is therefore an important one based on at least two different interpretations. From the point of view of other vertices, it is a highly sought-after hop for reaching others as quickly as possible. The second possible interpretation is that v itself is the best-situated vertex to reach others as quickly as possible.

The second reason for presenting the betweenness centrality as a success metric is its quantifiability. It is part of the HPC Scalable Graph Analysis Benchmarks (formerly known as the HPCS Scalable Synthetic Compact Applications #2 (Bader et al., a)) and various implementations on different platforms exist (Bader and Madduri, 2006a; Madduri et al., 2009; Tan et al., 2009) for comparison.

5.2 BC algorithm and experimental setup

We compute betweenness centrality using the algorithm of Brandes (2001). It computes single source shortest paths from each node in the network and increases the respective BC score for nodes on the path. The algorithm requires $O(nm)$ time for unweighted graphs and $O(nm + n^2 \log n)$ time for weighted graphs, where n is the number of nodes and m is the number of edges in the graph. The sizes of real-world graphs make the exact $O(nm)$ calculation too expensive, so we resort to efficient approximations. Bader et al. (2007) propose an unbiased estimator of betweenness centrality that is based on sampling nodes from which to compute single-source shortest paths. The resulting scores approximate a uniformly scaled version of the actual betweenness centrality. We focus on unweighted graphs in this performance study.

Following the specification of the graph analysis benchmark (Bader et al., a), we use R-MAT matrices as inputs. We report the performance of the approximate algorithm with 8192 starting vertices. We measure performance using the Traversed Edges Per Second (TEPS) rate, which is an algorithmic performance count that is independent of the particular implementation. We randomly relabeled the vertices in the generated graph before storing it for subsequent runs. For reproducibility of results, we chose starting vertices using a deterministic process, specifically excluding isolated vertices whose selection would have boosted the TEPS scores artificially.

We implemented an array-based formulation of Brandes' algorithm due to Robinson (2011). A reference Matlab implementation is publicly available from the Graph Analysis webpage (Bader et al., b). The workhorse of the algorithm is a parallel breadth-first search that is performed from multiple source vertices. In Combinatorial BLAS, one step of the breadth-first search is implemented as the multiplication of the transpose of the adjacency matrix of the graph with a rectangular matrix $\mathbf{X}$, where the ith column of $\mathbf{X}$ represents the current frontier of the ith independent breadth-first search tree. Initially, each column of $\mathbf{X}$ has only one non-zero that represents the starting vertex of the breadth-first search. The tallying step is also implemented as an SpGEMM operation.

For the performance results presented in this section, we use a synchronous implementation of the Sparse SUMMA algorithm (Buluç and Gilbert, 2008a; Buluç, 2010), because it is the most portable SpGEMM implementation and relies only on simple MPI-1 features. The other Combinatorial BLAS primitives that are used for implementing the betweenness centrality algorithm are reductions along one dimension and elementwise operations for sparse/sparse, sparse/dense, and dense/sparse input pairs. The experiments are run on TACC's Lonestar cluster (University of Texas, 2011) which is composed of dual-socket dual-core nodes connected by an InfiniBand interconnect that has 1 GB/s point-to-point bandwidth. Each individual processor is an Intel Xeon 5100, clocked at 2.66 GHz. We used the recommended Intel C++ compilers (version

10.1), and the MVAPICH2 (version 1.2rc2) implementation of MPI.

5.2.1 Parallel strong scaling. Figure 4 shows how our implementation scales for graphs of smaller size. Figure 5 shows the same code on larger graphs, with larger numbers of processors. Both results show good scaling for this challenging tightly-coupled algorithm. To the best of our knowledge, our results were the first distributed memory performance results for betweenness centrality on unweighted graphs (Buluç, 2010). Shortly afterwards, Edmonds et al. (2010) gave a coarse-grained algorithm for the general case, scaling to 64 processors on both weighted and unweighted graphs. Our results, in contrast, scale beyond thousands of processors, but only address the unweighted case.

The performance results on more than 500 processors are not smooth, but the overall upward trend is clear. Run time variability of large-scale parallel codes, which can be due to various factors such as OS jitter (Petrini et al., 2003), is widely reported in the literature (Van Straalen et al., 2009). The expensive computation prohibited us from running more experiments which would have smoothed out the results by averaging.

The best reported performance results for this problem are due to Madduri et al. (2009), who used an optimized implementation tailored for massively multithreaded architectures. They report a maximum of 160 million TEPS for an R-MAT graph of scale 24 on the 16-processor XMT machine. On the MTA-2 machine, which is the predecessor to the XMT, the same optimized code achieved 353 million TEPS on 40 processors. Our code, on the other hand, is truly generic and contains no problem or machine-specific optimizations. We did not optimize our primitives for the skewed aspect ratio (ratio of dimensions) of most of the matrices involved. For this problem instance, 900 processors of Lonestar were equivalent to 40 processors of MTA-2.

5.2.2 Sensitivity to batch processing. Most of the parallelism comes from the coarse-grained SpGEMM operation that is used to perform breadth-first searches from multiple source vertices. By changing the *batchsize*, the number of source vertices that are processed together, we can trade off space usage and potential parallelism. Space increases linearly with increasing batchsize. As we show experimentally, performance also increases substantially, especially for large numbers of processors. In Figure 6, we show the strong scaling of our betweenness centrality information on an R-MAT graph of scale 22 (approximately 4 million vertices and 32 million edges), using different batchsizes. The average performance gain of using 256, 512 and 1024 starting vertices, over using 128 vertices, is 18.2%, 29.0%, and 39.7%, respectively. The average is computed over the performance on $p = \{196, 225, \ldots, 961\}$ (perfect squares) processors. For larger numbers of processors, the performance gain of using a large batchsize is more substantial. For example, for $p = 961$, the performance increases by 40.4%, 67.0%, and 73.5% when using 256, 512 and 1024 starting vertices instead of 128.

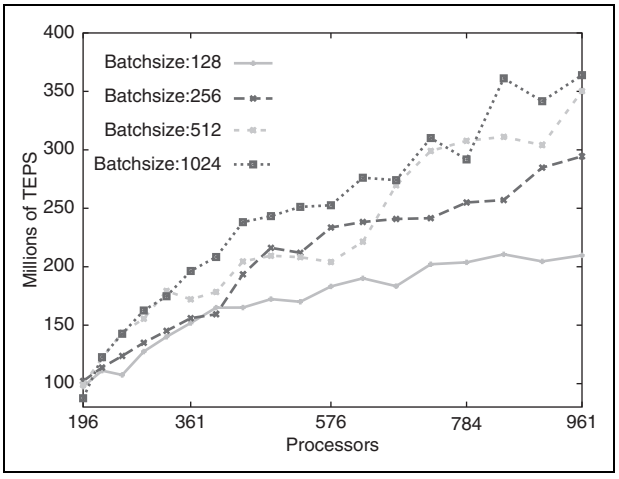


Figure 6. The effect of batch processing on the performance of the distributed-memory betweenness centrality implementation

5.3 Markov clustering

Markov clustering (MCL) (Van Dongen, 2008) is a flow-based graph clustering algorithm that has been popular in computational biology, among other fields. It simulates a Markov process to the point where clusters can be identified by a simple interpretation of the modified adjacency matrix of the graph. Computationally, it alternates between an expansion step in which the adjacency matrix is raised to its nth power (typically $n = 2$), and an inflation step in which the scalar entries are raised to the dth power ($d > 1$) and then renormalized within each column. The inflation operation boosts the larger entries and sends the smaller entries closer to zero. MCL maintains sparsity of the matrix by pruning small entries after the inflation step.

Implementing the MCL algorithm using the Combinatorial BLAS primitives generates a naturally concise code. The full MCL code, except for the cluster interpretation, is shown in Figure 8; the inflation subroutine is shown in Figure 7.

Van Dongen (2000) provides a fast sequential implementation of the MCL algorithm. We do not attempt an apples-to-apples comparison with the original implementation, as that software has many options which we do not replicate in our 10–15 line prototype. Van Dongen's sequential **mcl** code is twice as fast as our parallel implementation on a single processor. This is mostly due to its finer control over sparsity parameters, such as limiting the number of non-zeros in each row and column. However, serial performance is not a bottleneck, as our code achieves superlinear speedup until $p = 1024$.

We have been able to cluster gigascale graphs with our implementation of MCL using the Combinatorial BLAS. Here, we report on a smaller instance in order to provide a complete strong scaling result. Figure 9 shows the speedup of the three most expensive iterations, which

```
template <typename IT, typename NT, typename DER>
void Inflate(SpParMat<IT,NT,DER> & A, double power)
{
    A.Apply(bind2nd(exponentiate(), power));

    /* reduce to Row, columns are collapsed to single entries */
    DenseParVec<IT,NT> colsums = Reduce(Row, plus<NT>(), NT());

    /* multinv<NT>() functor is user defined for type NT */
    colsums.Apply(multinv<NT>());

    /* scale each Column with the given vector */
    A.DimScale(colsums, Column);
}
```

Figure 7. Inflation code using the Combinatorial BLAS primitives

```
int main()
{
    SpParMat<unsigned, double, SpDCCols<unsigned, double> > A;
    A.ReadDistribute(''inputmatrix'');

    oldchaos = Chaos(A);
    newchaos = oldchaos;

    // while there is an epsilon improvement
    while((oldchaos - newchaos) > EPS)
    {
        A.Square(); // expand
        Inflate(A,2);    // inflate and renormalize
        A.Prune(bind2nd(less<double>(), 0.0001));
        oldchaos = newchaos;
        newchaos = Chaos(A);
    }
    Interpret(A);
}
```

Figure 8. MCL code using the Combinatorial BLAS primitives

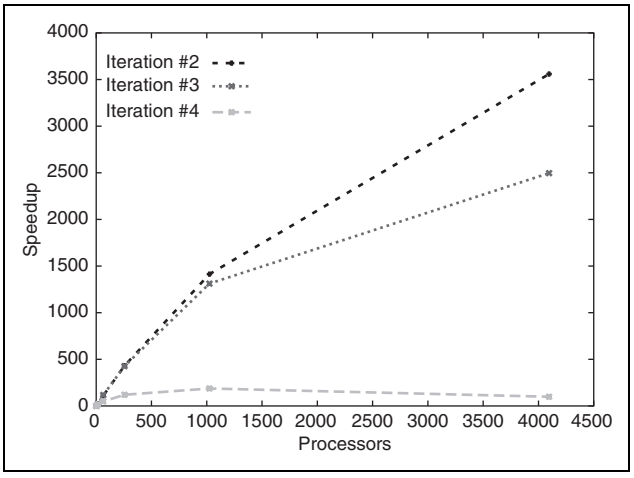


Figure 9. Strong scaling of the three most expensive iterations while clustering an R-MAT graph of scale 14 using the MCL algorithm implemented using the Combinatorial BLAS

together make up more than 99% of the total running time. The input is a permuted R-MAT graph of scale 14, with self loops added. On 4096 processors we were able to cluster this graph in less than a second. The same graph takes more than half an hour to cluster on a single processor. Note that iteration #4 takes only 70 milliseconds using 1024 processors, which is hard to scale further due to parallelization overheads on thousands of processors.

6 Conclusions and future work

Linear algebra has played a crucial role as the middleware between continuous physical models and their computer implementation. We have introduced the Combinatorial BLAS library as the middleware between discrete structures and their computer implementation. To accommodate future extensions and developments, we have avoided

explicit specifications and focused on guiding principles instead.

The Combinatorial BLAS aggregates elementary operations to a level that allows optimization and load-balancing within its algebraic primitives. Our efficient parallel sparse array infrastructure, which uses a 2D compressed sparse block data representation, provides efficiency and scalability, as demonstrated by large scale experiments on two important graph applications.

Our MPI implementation, albeit portable due to the widespread adoption of MPI, does not take advantage of flexible shared-memory operations. Part of our future work will be to leverage the hierarchical parallelism that is characteristic of current and future supercomputers. The architecture of the Combinatorial BLAS is flexible enough to accommodate drop-in shared-memory replacements for sequential routines, without altering distributed objects and algorithms. For certain primitives, such as SpGEMM, efficient shared-memory algorithms are unknown at this point. We are actively working on developing fast shared-memory algorithms for those primitives, and we will support hybrid parallelism within the Combinatorial BLAS once those algorithmic innovations are done.

Acknowledgment

Part of this work was performed while Aydın Buluç was with the University of California, Santa Barbara.

Funding

This work was supported in part by the National Science Foundation (grant number CRI-IAD0709385) and through TeraGrid resources provided by TACC (grant number TG-CCR090036).

Conflict of interest

None declared.

References

Ajwani, D., Meyer, U. and Osipov, V. (2007). Improved external memory BFS implementation. In: *ALENEX*, SIAM.

Anderson, E., Bai, Z., Bischof, C., Demmel, J., Dongarra, J., Du Croz, J. et al. (1992). *LAPACK's User's Guide*. Philadelphia, PA, USA: Society for Industrial and Applied Mathematics.

Asanovic, K., Bodik, R., Catanzaro, B., Gebis, J., Husbands, P., Keutzer, K. et al. (2006). The landscape of parallel computing research: A view from Berkeley. Technical Report UCB/EECS-2006-183 EECS Department, University of California at Berkeley. http://www.eecs.berkeley.edu/Pubs/TechRpts/2006/EECS-2006-183.html.

Bader, D., Feo, J., Gilbert, J., Kepner, J., Koester, D., Loh, E. et al. (a) HPCS scalable synthetic compact applications #2. Version 1.1. http://www.highproductivity.org/SSCABmks.htm.

Bader, D., Gilbert, J., Kepner, J. and Madduri, K. (b) HPC graph analysis benchmark. http://www.graphanalysis.org/benchmark.

Bader, D. A., Kintali, S., Madduri, K. and Mihail, M. (2007). Approximating betweenness centrality. *Lecture Notes in Computer Science* **4863**: 124–137.

Bader, D. and Madduri, K. (2006a).. Parallel algorithms for evaluating centrality indices in real-world networks. In: *Proceedings of the 35th International Conference on Parallel Processing (ICPP 2006)*, Washington, DC, USA: IEEE Computer Society, pp. 539–550.

Bader, D. A., and Madduri, K. (2006b). Designing multithreaded algorithms for breadth-first search and st-connectivity on the Cray MTA-2. In: *Proceedings of the 35th International Conference on Parallel Processing (ICPP 2006)*, Washington, DC, USA: IEEE Computer Society, pp. 523–530.

Bader, D. A., and Madduri, K. (2008). SNAP, Small-world Network Analysis and Partitioning: An open-source parallel graph framework for the exploration of large-scale networks. In: *IPDPS'08: Proceedings of the 2008 IEEE International Symposium on Parallel & Distributed Processing*, pp. 1–12.

Balay, S., Gropp, W. D., McInnes, L. C., and Smith, B. F. (1997). Efficient management of parallelism in object oriented numerical software libraries. In: Arge E, Bruaset AM and Langtangen HP (eds.) *Modern Software Tools in Scientific Computing*. Birkhäuser Press, pp. 163–202.

Barrett, B. W., Berry, J. W., Murphy, R. C., and Wheeler, K. B., (2009). Implementing a portable multi-threaded graph library: The MTGL on Qthreads. In: *IPDPS'09: Proceedings of the 2009 IEEE International Symposium on Parallel & Distributed Processing*, pp. 1–8.

Barton, J. J., and Nackman, L. R. (1994). *Scientific and Engineering C++: An Introduction with Advanced Techniques and Examples*. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc.

Berry, J. W., Hendrickson, B., Kahan, S., and Konecny, P. (2007). Software and algorithms for graph queries on multithreaded architectures. In: *IPDPS'07: Proceedings of the 2007 IEEE International Symposium on Parallel & Distributed Processing*, pp. 1–14.

Blelloch, GE., (1990). *Vector Models for Data-Parallel Computing*. Cambridge, MA, USA: MIT Press.

Bonachea, D. (2002). GASNet specification, v1.1. Technical Report CSD-02-1207, Computer Science Division, University of California Berkeley.

Brandes, U. (2001). A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology* 25: 163–177.

Briggs, W. L., Henson, V. E., and McCormick, S. F. (2000). *A Multigrid Tutorial: Second Edition*. Philadelphia, PA, USA: Society for Industrial and Applied Mathematics.

Brodman, J. C., Fraguela, B. B., Garzarán, M. J., and Padua, D. (2009). New abstractions for data parallel programming. In: *HotPar'09: Proceedings of the 1st USENIX Workshop on Hot Topics in Parallelism*.

Buluç, A. (2010). *Linear Algebraic Primitives for Parallel Computing on Large Graphs*. PhD thesis, University of California, Santa Barbara.

Buluç A, Fineman JT, Frigo M, Gilbert JR and Leiserson CE (2009). Parallel sparse matrix–vector and matrix-transpose–

vector multiplication using compressed sparse blocks. In: *Proceedings of the 21st ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pp. 233–244.

Buluç, A., and Gilbert, J. R. (2008a). Challenges and advances in parallel sparse matrix–matrix multiplication. In: *ICPP'08: Proceedings of the International Conference on Parallel Processing*, Portland, Oregon, USA: IEEE Computer Society, pp. 503–510.

Buluç, A., and Gilbert, J. R. (2008b). On the representation and multiplication of hypersparse matrices. In: *IPDPS'08: Proceedings of the 2008 IEEE International Symposium on Parallel & Distributed Processing*, pp. 1–11.

Buluç, A., and Gilbert, J. R. (2010). Highly parallel sparse matrix–matrix multiplication. Technical Report UCSB-CS-2010-10, Computer Science Department, University of California Santa Barbara. http://arxiv.org/abs/1006.2183.

Burrus, N., Duret-Lutz, A., Duret-Lutz, R., Geraud, T., Lesage, D., and Poss, R. (2003). A static C++ object-oriented programming (SCOOP) paradigm mixing benefits of traditional OOP and generic programming. In: *Proceedings of the Workshop on Multiple Paradigm with OO Languages (MPOOL)*.

Cardelli, L., and Wegner, P. (1985). On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys* 17: 471–523.

Cohen, J. (2009). Graph twiddling in a MapReduce world. *Computing in Science and Engineering* 11: 29–41.

Coplien, J. O. (1995). Curiously recurring template patterns. *C++ Report* 7: 24–27.

Dean, J., and Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. *Communications of the ACM* 51: 107–113.

Dongarra, J. (2000). Sparse matrix storage formats. In: Bai Z, Demmel J, Dongarra J, Ruhe A and van der Vorst H (eds.) *Templates for the Solution of Algebraic Eigenvalue Problems: A Practical Guide*. Philadelphia, PA: SIAM.

Duff, I. S., Heroux, M. A., and Pozo, R. (2002). An overview of the Sparse Basic Linear Algebra Subprograms: The new standard from the BLAS technical forum. *ACM Transactions on Mathematical Software* 28: 239–267.

Edmonds, N., Hoefler, T., and Lumsdaine, A. (2010). A Space-Efficient Parallel Algorithm for Computing Betweenness Centrality in Distributed Memory. Accepted at the 2010 International Conference on High Performance Computing (HiPC'10).

Fineman, J., and Robinson, E. (2011). Fundamental graph algorithms. In: Kepner J and Gilbert J (eds.) *Graph Algorithms in the Language of Linear Algebra*. Philadelphia, PA: SIAM.

Freeman, L. C. (1977). A set of measures of centrality based on betweenness. *Sociometry* 40: 35–41.

Gilbert, J. R., Reinhardt, S., and Shah, V. B. (2008). A unified framework for numerical and combinatorial computing. *Computing in Science and Engineering* 10: 20–25.

Grama, A., Karypis, G., Gupta, A., and Kumar, V. (2003). *Introduction to Parallel Computing: Design and Analysis of Algorithms*. Addison-Wesley.

Gregor, D., and Lumsdaine, A. (2005). The Parallel BGL: A generic library for distributed graph computations. In: *Workshop on Parallel Object-Oriented Scientific Computing (POOSC)*.

Heroux, M. A., Bartlett, R. A., Howle, V. E., Hoekstra, R. J., Hu, J. J., Kolda, T. G. et al. (2005). An overview of the Trilinos project. *ACM Transactions on Mathematical Software* 31: 397–423.

Järvi, J., Willcock, J., and Lumsdaine, A. (2003). Concept-controlled polymorphism. In: *Proceedings of the 2nd International Conference on Generative Programming and Component Engineering* New York, NY, USA: Springer-Verlag, pp. 228–244.

Kepner, J. (2009). *Parallel MATLAB for Multicore and Multinode Computers*. Philadelphia, PA, USA: Society for Industrial and Applied Mathematics.

Kirsch, C., Payer, H. and Röck H (2010). Scal: Non-linearizable computing breaks the scalability barrier. Technical Report 2010-07, Department of Computer Sciences, University of Salzburg.

Lawson, C. L., Hanson, R. J., Kincaid, D. R., and Krogh, F. T. (1979). Basic linear algebra subprograms for Fortran usage. *ACM Transactions on Mathematical Software* 5: 308–323.

Lumsdaine, A., Gregor, D., Hendrickson, B., and Berry, J. (2007). Challenges in parallel graph processing. *Parallel Processing Letters* 17: 5–20.

Madduri, K., Ediger, D., Jiang, K., Bader, D. and Chavarria-Miranda D (2009). A faster parallel algorithm and efficient multithreaded implementations for evaluating betweenness centrality on massive datasets. In: *Proceedings of the 3rd Workshop on Multithreaded Architectures and Applications (MTAAP 2009)*. IEEE Computer Society.

Malewicz, G., Austern, M. H., Bik, A. J., Dehnert, J. C., Horn, I., Leiser, N. et al. (2010). Pregel: a system for large-scale graph processing. In: *Proceedings of the 2010 International Conference on Management of Data* New York, NY, USA, pp. 135–146.

Nieplocha, J., Palmer, B., Tipparaju, V., Krishnan, M., Trease, H., and Aprà, E. (2006). Advances, applications and performance of the global arrays shared memory programming toolkit. *International Journal of High Performance Computing Applications* 20: 203–231.

Nieplocha, J., Tipparaju, V., Krishnan, M., and Panda, D. K. (2005). High performance remote memory access communication: The ARMCI approach. *International Journal of High Performance Computing Applications* 20: 233–253.

Petrini, F., Kerbyson, D. J., and Pakin, S. (2003). The case of the missing supercomputer performance: Achieving optimal performance on the 8192 processors of ASCI Q. In: *SC '03: Proceedings of the 2003 ACM/IEEE Conference on Supercomputing* Washington, DC, USA: IEEE Computer Society, p. 55.

Robinson, E. (2011). Complex graph algorithms. In: Kepner, J., and Gilbert, J. (eds.) *Graph Algorithms in the Language of Linear Algebra*. Philadelphia, PA: SIAM.

Saad, Y. (2003). *Iterative Methods for Sparse Linear Systems* (2nd edition). Philadelpha, PA: SIAM.

Shah, V., and Gilbert, J. R. (2004). Sparse matrices in Matlab∗P: Design and implementation. *Lecture Notes in Computer Science* 3296: 144–155. URL http://gauss.cs.ucsb.edu/publication/dsparse.pdf.

Siek, J. G., Lee, L. Q., and Lumsdaine, A. (2001). *The Boost Graph Library User Guide and Reference Manual (With CD-ROM)*. Addison-Wesley Professional.

Tan, G., Sreedhar, V., and Gao, G. (2009). Analysis and performance results of computing betweenness centrality on IBM Cyclops64. *The Journal of Supercomputing* : 1–24.

University of Texas (2011). *Lonestar User Guide*. http://services.tacc.utexas.edu/index.php/lonestar-user-guide.

Valiant, L. G. (1990). A bridging model for parallel computation. *Communications of the ACM* 33: 103–111.

Van Dongen, S. (2000). MCL – a cluster algorithm for graphs. http://www.micans.org/mcl/index.html.

Van Dongen, S. (2008). Graph clustering via a discrete uncoupling process. *SIAM Journal on Matrix Analysis and Applications* 30: 121–141.

Van Straalen, B., Shalf, J., Ligocki, T., Keen, N., and Yang, W.S. (2009). Scalability challenges for massively parallel AMR applications. In: *IPDPS '09: Proceedings of the 2009 IEEE International Symposium on Parallel & Distributed Processing* Washington, DC, USA: IEEE Computer Society, pp. 1–12.

Author's Biographies

Aydın Buluç is a Luis W Alvarez Postdoctoral Fellow at Lawrence Berkeley National Laboratory. His research interests include parallel computing, combinatorial scientific computing, high performance graph analysis and sparse matrix computations. He received his PhD in Computer Science from the University of California, Santa Barbara, in 2010 and his BS in Computer Science and Engineering from Sabanci University, Turkey, in 2005. As a graduate student, he spent a semester at the Mathematics Department of MIT, and a summer at the CSRI institute of Sandia National Labs, in New Mexico. He is a member of the SIAM and the ACM.

John R Gilbert received his PhD in Computer Science at Stanford in 1981. From 1981 to 1988 he was on the Computer Science faculty at Cornell. In 1988 he moved to Xerox PARC, where he performed and directed research in parallel computing, computational geometry, languages and compilers for high-performance computing, and mathematical algorithms and software. In 1997 he founded the Computation and Matter Area at PARC, whose projects included distributed data analysis for collaborating sensors, meso-scale MEMS for active surfaces, and modular robotics. In 2002, Dr Gilbert joined the Computer Science Department and the Computational Science and Engineering program at the University of California, Santa Barbara, where he leads research in high-performance computing, combinatorial scientific computing, and software for discrete and numerical computation. Dr Gilbert is a Fellow of the Society for Industrial and Applied Mathematics. He has served on the SIAM Council, chaired the SIAM Activity Group on Supercomputing and the ACM Special Interest Group on Numerical Mathematics, and served on the editorial boards of several journals in computational science and applied mathematics.